

VQC Hyperparameter-Analyse - Optimizer & Iterationen

Modell: VQC (RealAmplitudes)

Datensatz: Iris (3 Klassen, 2 Features, fair) - identische Pipeline wie 05_vergleich_iris_fair.ipynb

Analyse: 3 Optimizer × 4 Iterationsstufen = 12 Läufe

Ausgabe: ergebnisse_hyperparam.csv (separat von ergebnisse.csv)

Optimizer	Typ
COBYLA	Gradientenfrei, simplex-basiert
SPSA	Gradientenfrei, stochastisch, rauschrobust
ADAM	Gradientenbasiert, adaptiv

0. Imports

```
In [1]: import sys, os
sys.path.append(os.path.abspath(".."))

import numpy as np
import pandas as pd
import time
import warnings
warnings.filterwarnings("ignore")

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import accuracy_score, f1_score

from qiskit.circuit.library import zz_feature_map, real_amplitudes
from qiskit_machine_learning.algorithms import VQC
from qiskit.primitives import StatevectorSampler
from qiskit_machine_learning.optimizers import COBYLA, SPSA, ADAM

print("Imports OK")
```

Imports OK

1. Datensatz - identische Pipeline

Exakt gleiche Konfiguration wie 02_iris_fair.ipynb.

```
In [2]: iris = load_iris()
X = iris.data[:, [0, 2]]
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

scaler = MinMaxScaler(feature_range=(0, 2 * np.pi))
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

print(f"Train: {X_train_sc.shape} | Test: {X_test_sc.shape}")
```

Train: (105, 2) | Test: (45, 2)

2. Analyse-Loop

12 Läufe: 3 Optimizer × 4 Iterationsstufen (50, 100, 150, 200).

Ergebnisse werden laufend in `ergebnisse_hyperparam.csv` geschrieben.

```
In [3]: CSV_PATH = "Ergebnisse/ergebnisse_hyperparam.csv"

OPTIMIZERS = {
    "COBYLA": lambda n: COBYLA(maxiter=n),
    "SPSA":   lambda n: SPSA(maxiter=n),
    "ADAM":   lambda n: ADAM(maxiter=n),
}

ITERATIONEN = [50, 100, 150, 200]

results = []

total = len(OPTIMIZERS) * len(ITERATIONEN)
run = 0

for opt_name, opt_fn in OPTIMIZERS.items():
    for iters in ITERATIONEN:
        run += 1
        print(f"[{run}/{total}] {opt_name} - {iters} Iterationen ...", end="

        feature_map = zz_feature_map(feature_dimension=2, reps=1)
        ansatz = real_amplitudes(num_qubits=2, reps=2)

        vqc = VQC(
            feature_map=feature_map,
            ansatz=ansatz,
            optimizer=opt_fn(iters),
            sampler=StatevectorSampler(),
        )

        start = time.time()
        vqc.fit(X_train_sc, y_train)
        train_time = round(time.time() - start, 4)
```

```

start = time.time()
y_pred = vqc.predict(X_test_sc)
infer_time = round(time.time() - start, 4)

acc = round(accuracy_score(y_test, y_pred), 4)
f1 = round(f1_score(y_test, y_pred, average="weighted"), 4)

print(f"Accuracy: {acc:.4f} F1: {f1:.4f} ({train_time}s)")

entry = {
    "Modell": "VQC (RealAmplitudes)",
    "Datensatz": "Iris (3 Klassen, 2 Features, fair)",
    "Backend": "AerSimulator",
    "Optimizer": opt_name,
    "Iterationen": iters,
    "Accuracy": acc,
    "F1": f1,
    "Trainingszeit_s": train_time,
    "Inferenzzeit_s": infer_time,
}
results.append(entry)

# Laufend speichern - bei Absturz gehen keine Daten verloren
df_entry = pd.DataFrame([entry])
if os.path.exists(CSV_PATH):
    df_entry.to_csv(CSV_PATH, mode="a", header=False, index=False)
else:
    df_entry.to_csv(CSV_PATH, index=False)

print()
print("Alle Läufe abgeschlossen.")

```

[1/12] COBYLA - 50 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.5111 F1: 0.4057 (38.0302s)

[2/12] COBYLA - 100 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.5556 F1: 0.4467 (47.5039s)

[3/12] COBYLA - 150 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.5556 F1: 0.4444 (41.9784s)

[4/12] COBYLA - 200 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.4444 F1: 0.3595 (48.2921s)

[5/12] SPSA - 50 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.5111 F1: 0.4075 (112.2252s)

[6/12] SPSA - 100 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.5556 F1: 0.4444 (189.1446s)

[7/12] SPSA - 150 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.5333 F1: 0.4254 (261.4977s)

[8/12] SPSA - 200 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.5111 F1: 0.4057 (337.9421s)

[9/12] ADAM - 50 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.3333 F1: 0.2645 (611.9295s)

[10/12] ADAM - 100 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.3778 F1: 0.3001 (1227.8034s)

[11/12] ADAM - 150 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.3778 F1: 0.3025 (1861.0865s)

[12/12] ADAM - 200 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.3111 F1: 0.2476 (2444.515s)

Alle Läufe abgeschlossen.

3. Ergebnisübersicht

```
In [4]: df = pd.read_csv(CSV_PATH)
print(df[["Optimizer", "Iterationen", "Accuracy", "F1", "Trainingszeit_s"]])
```

Optimizer	Iterationen	Accuracy	F1	Trainingszeit_s
COBYLA	50	0.5111	0.4057	38.0302
COBYLA	100	0.5556	0.4467	47.5039
COBYLA	150	0.5556	0.4444	41.9784
COBYLA	200	0.4444	0.3595	48.2921
SPSA	50	0.5111	0.4075	112.2252
SPSA	100	0.5556	0.4444	189.1446
SPSA	150	0.5333	0.4254	261.4977
SPSA	200	0.5111	0.4057	337.9421
ADAM	50	0.3333	0.2645	611.9295
ADAM	100	0.3778	0.3001	1227.8034
ADAM	150	0.3778	0.3025	1861.0865
ADAM	200	0.3111	0.2476	2444.5150

4. Pivot-Tabelle - Accuracy

```
In [5]: pivot_acc = df.pivot(index="Optimizer", columns="Iterationen", values="Accuracy")
print("Accuracy:")
print(pivot_acc.to_string())
print()
pivot_f1 = df.pivot(index="Optimizer", columns="Iterationen", values="F1")
print("F1:")
print(pivot_f1.to_string())
```

Accuracy:

Iterationen	50	100	150	200
Optimizer				
ADAM	0.3333	0.3778	0.3778	0.3111
COBYLA	0.5111	0.5556	0.5556	0.4444
SPSA	0.5111	0.5556	0.5333	0.5111

F1:

Iterationen	50	100	150	200
Optimizer				
ADAM	0.2645	0.3001	0.3025	0.2476
COBYLA	0.4057	0.4467	0.4444	0.3595
SPSA	0.4075	0.4444	0.4254	0.4057

5. Pivot-Tabelle - Trainingszeit

```
In [6]: pivot_time = df.pivot(index="Optimizer", columns="Iterationen", values="Trainingszeit (s)")
print("Trainingszeit (s):")
print(pivot_time.to_string())
```

Trainingszeit (s):

Iterationen	50	100	150	200
Optimizer				
ADAM	611.9295	1227.8034	1861.0865	2444.5150
COBYLA	38.0302	47.5039	41.9784	48.2921
SPSA	112.2252	189.1446	261.4977	337.9421